# link-state routing

will keep idea of sharing state with neighbors…

but weren't sharing enough state!

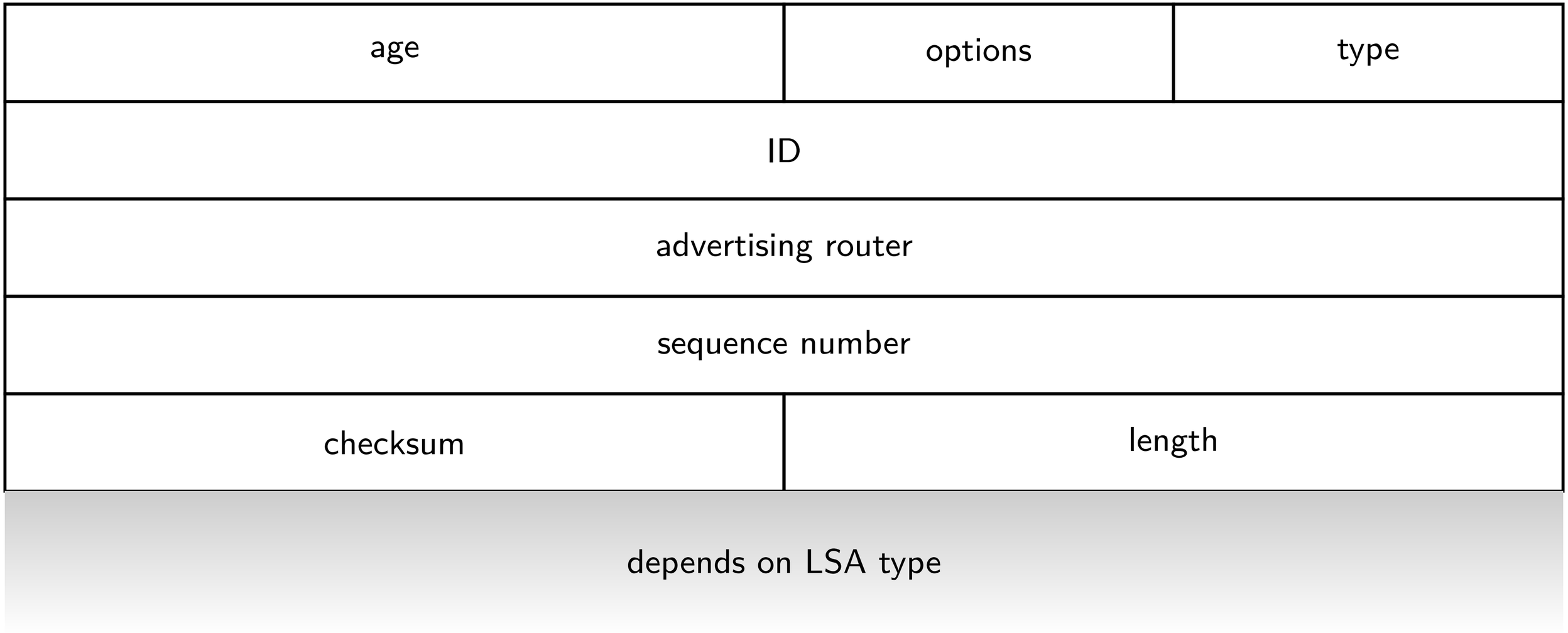
other routing idea:

routers collect *complete map* of network

example protocol for this: OSPF

Open Shortest Path First

# OSPF link-state advertisements (router)



# OSPF LSA sequences/ages

sequence number for getting correction version of LSAs

- some tricky rules to handle routers restarting (losing track of sequence number) and sequence number wraparound

maximum ‘age’ for link-state advertisements

- typically minutes
- too-old LSAs not used for routing
- deliberately setting age = MaxAge used to invalidate LSAs

# OSPF LSA types

‘router’:

- list of links for router
- links = connect to other router or network
- links refer to ID numbers of network/router LSAs
- metrics for each link

‘network’:

- list of routers for network
- different version of external and internal networks

(later) ‘summary’:

- part of support for *areas*
- used when sysadmin doesn’t want all routers processing whole network map

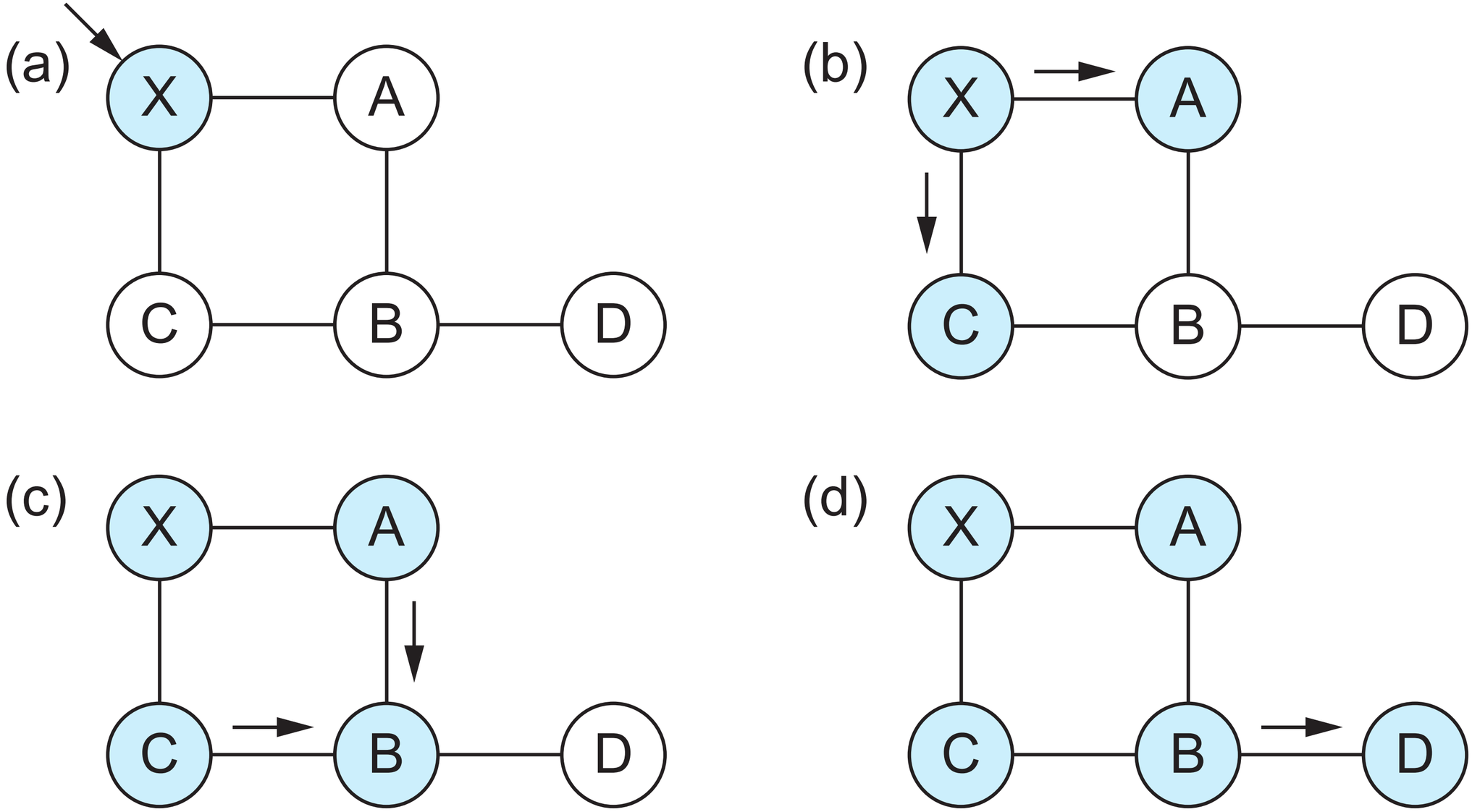
# link-state database

whole collection of advertisements

every router, network, link between router+network

all the metrics for those

# reliable flooding (picture)




Peterson and Davie, *Computer Networks: A Systems Approach*, Figure 88

# missing from the picture

in picture: seems like each router directly connected to each other

often we have multiple routers connected to local network

can/will share link state packets by broadcasting on local network

# reliable flooding in OSPF — setup

for each subnetwork:

- choose a designated and backup router
- make sure backup becomes designated on failure

designated router will take care of propogating updates to everyone on network

… including waiting for acknowledgments, etc.

# reliable flooding in OSPF

then, when receiving/generating link state packet:

send to every designed+backup router of subnetwork that

- you are connected to, and
- you are not designated/backup router for, and
- you did not receive the packet from

send to every router on every subnetwork that

- you are designated router for

send = send + resend if no ACK

# finding shortest paths

given full picture of network

want to find all shortest paths from self

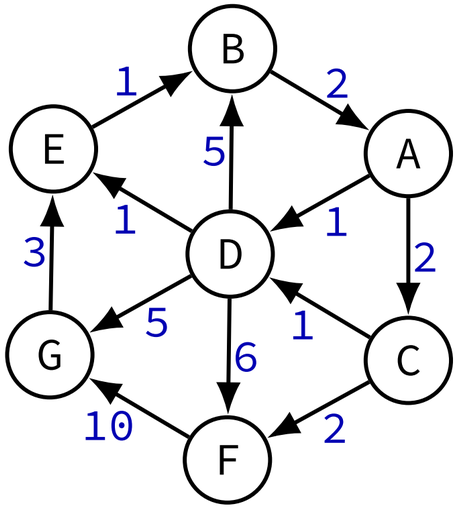
shortest ‘distance’ = lowest sum of metric

only need next hop, but will compute whole path to find that

assumption: everyone using shortest path

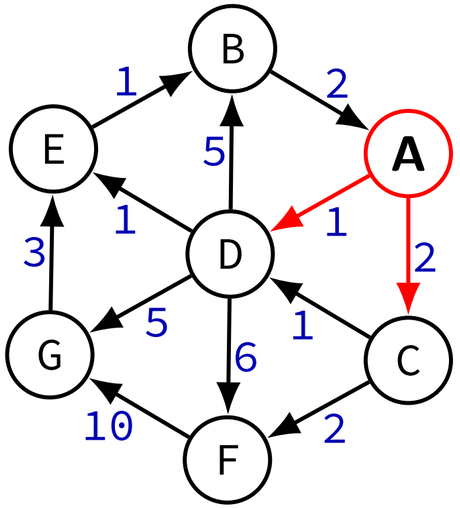
usual solution: Dijkstra’s algorithm

# Dijkstra's algorithm example 1



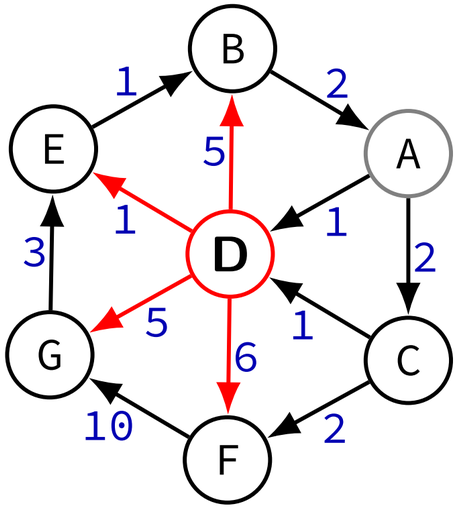

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | ∞ | — | — |
| C | ∞ | — | — |
| D | ∞ | — | — |
| E | ∞ | — | — |
| F | ∞ | — | — |
| G | ∞ | — | — |

# Dijkstra's algorithm example 1



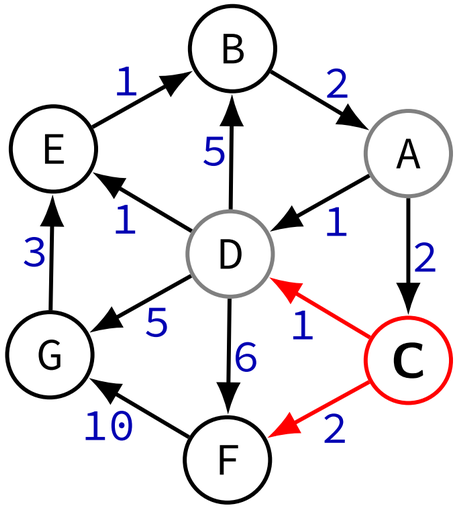

| | dist | prev | path |
|---|---|---|---|
| **A** | 0 | — | A |
| B | ∞ | — | — |
| **C** | 2 | A | A→C |
| **D** | 1 | A | A→D |
| E | ∞ | — | — |
| F | ∞ | — | — |
| G | ∞ | — | — |

# Dijkstra's algorithm example 1




| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| **B** | 6 | D | A→D→B |
| C | 2 | A | A→C |
| **D** | 1 | A | A→D |
| **E** | 2 | D | A→D→E |
| **F** | 7 | D | A→D→F |
| **G** | 6 | D | A→D→G |

# Dijkstra's algorithm example 1

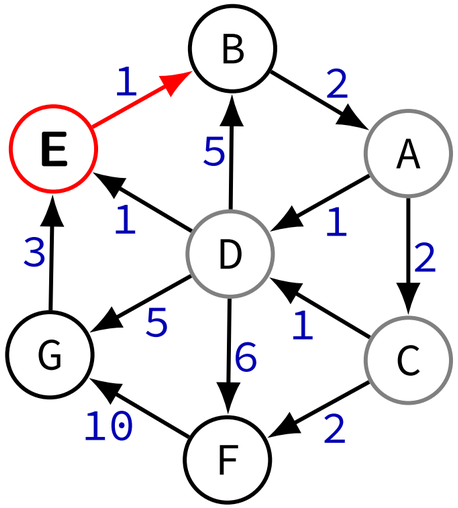



D is adjacent —
but not a shorter path

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 6 | D | A→D→B |
| **C** | 2 | A | A→C |
| D | 1 | A | A→D |
| E | 2 | D | A→D→E |
| **F** | 4 | C | A→C→F |
| G | 6 | D | A→D→G |

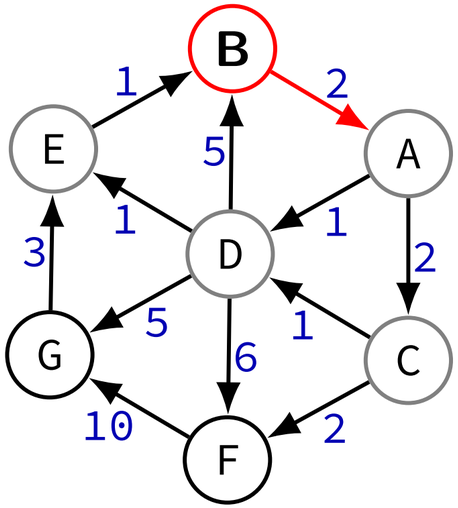
F updated from distance 7 (via D)
to distance 4 (via C)

# Dijkstra's algorithm example 1



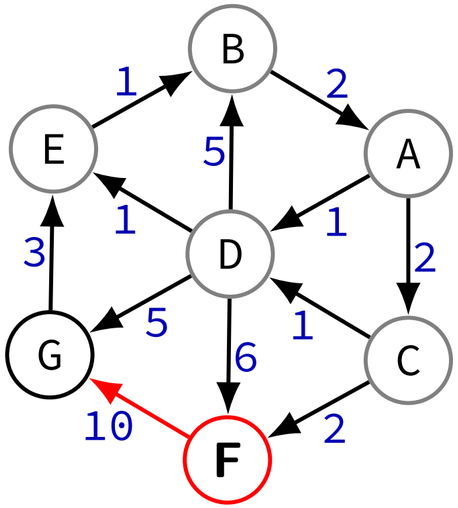

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| **B** | 3 | E | A→D→E→B |
| C | 2 | A | A→C |
| D | 1 | A | A→D |
| **E** | 2 | D | A→D→E |
| F | 4 | C | A→C→F |
| G | 6 | D | A→D→G |

# Dijkstra's algorithm example 1



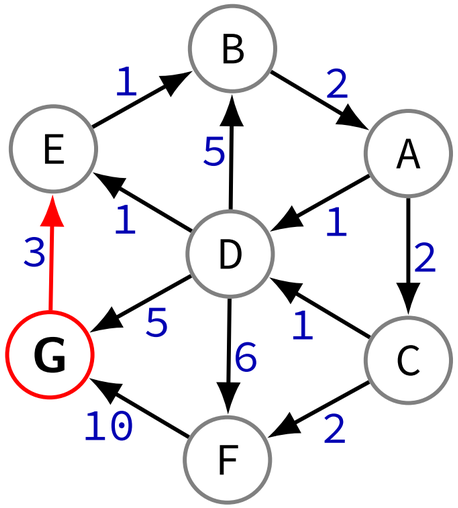

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| **B** | 3 | E | A→D→E→B |
| C | 2 | A | A→C |
| D | 1 | A | A→D |
| E | 2 | D | A→D→E |
| F | 4 | C | A→C→F |
| G | 6 | D | A→D→G |

# Dijkstra's algorithm example 1



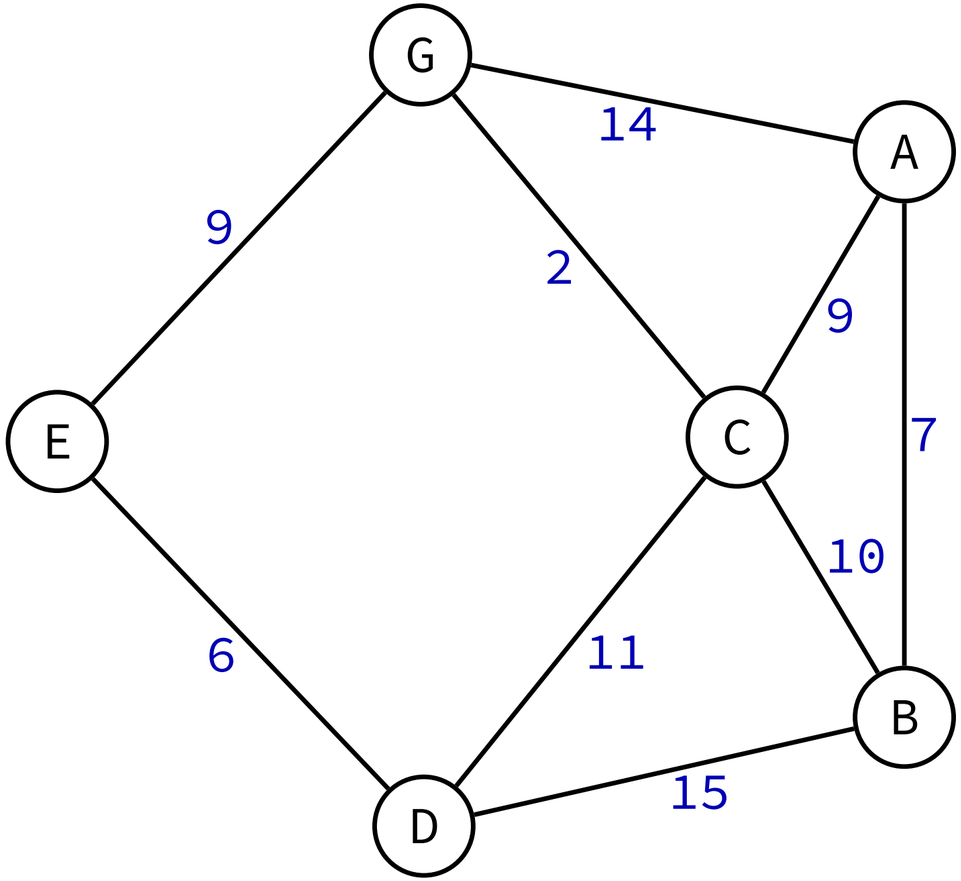

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 3 | E | A→D→E→B |
| C | 2 | A | A→C |
| D | 1 | A | A→D |
| E | 2 | D | A→D→E |
| **F** | 4 | C | A→C→F |
| G | 6 | D | A→D→G |

# Dijkstra's algorithm example 1



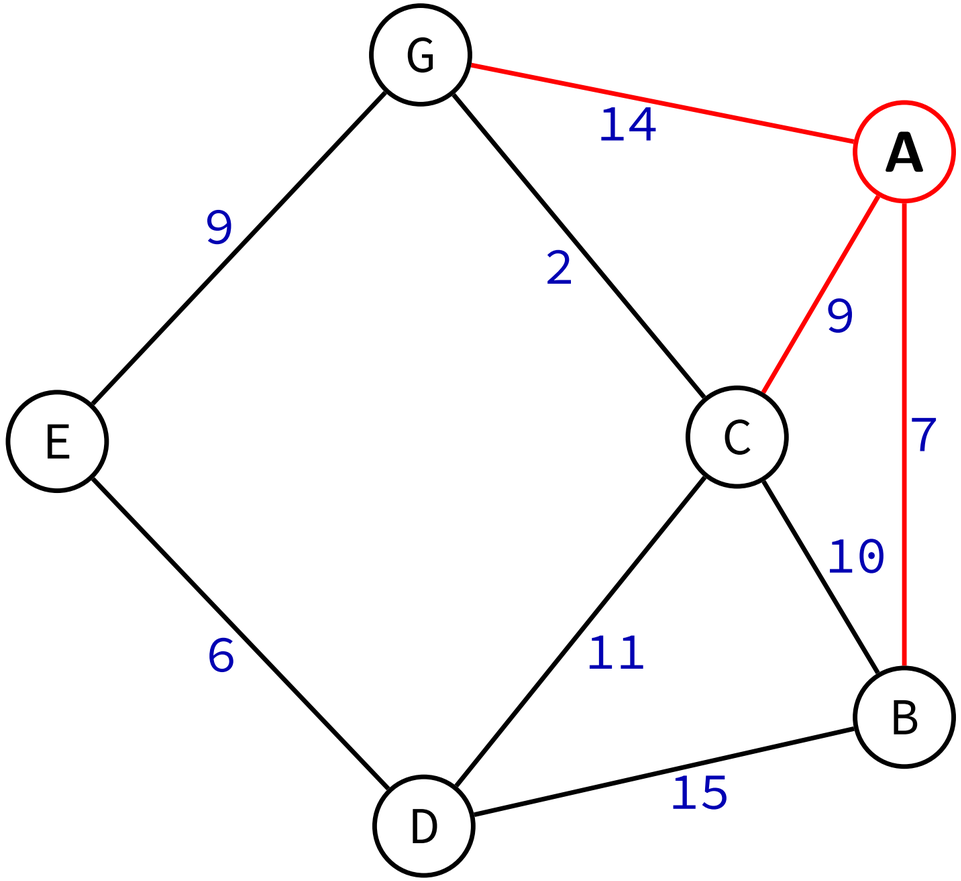

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 3 | E | A→D→E→B |
| C | 2 | A | A→C |
| D | 1 | A | A→D |
| E | 2 | D | A→D→E |
| F | 4 | C | A→C→F |
| **G** | 6 | D | A→D→G |

# Dijkstra's algorithm example 2



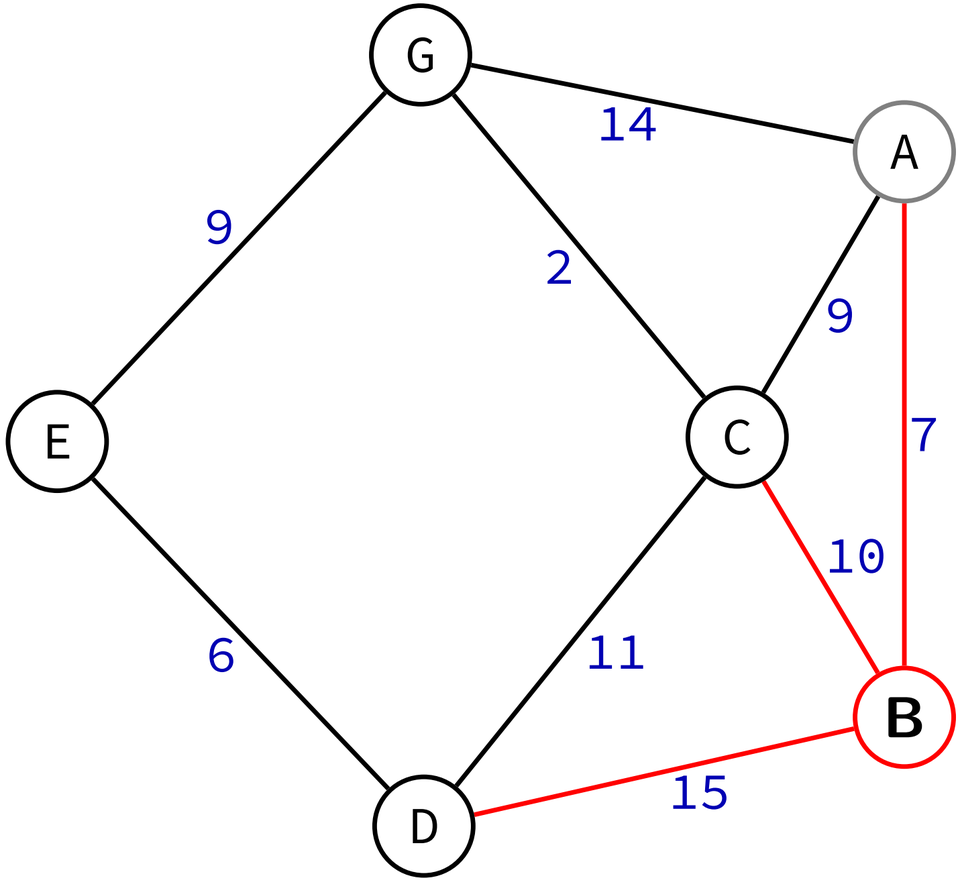

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | ∞ | — | — |
| C | ∞ | — | — |
| D | ∞ | — | — |
| E | ∞ | — | — |
| G | ∞ | — | — |

# Dijkstra's algorithm example 2



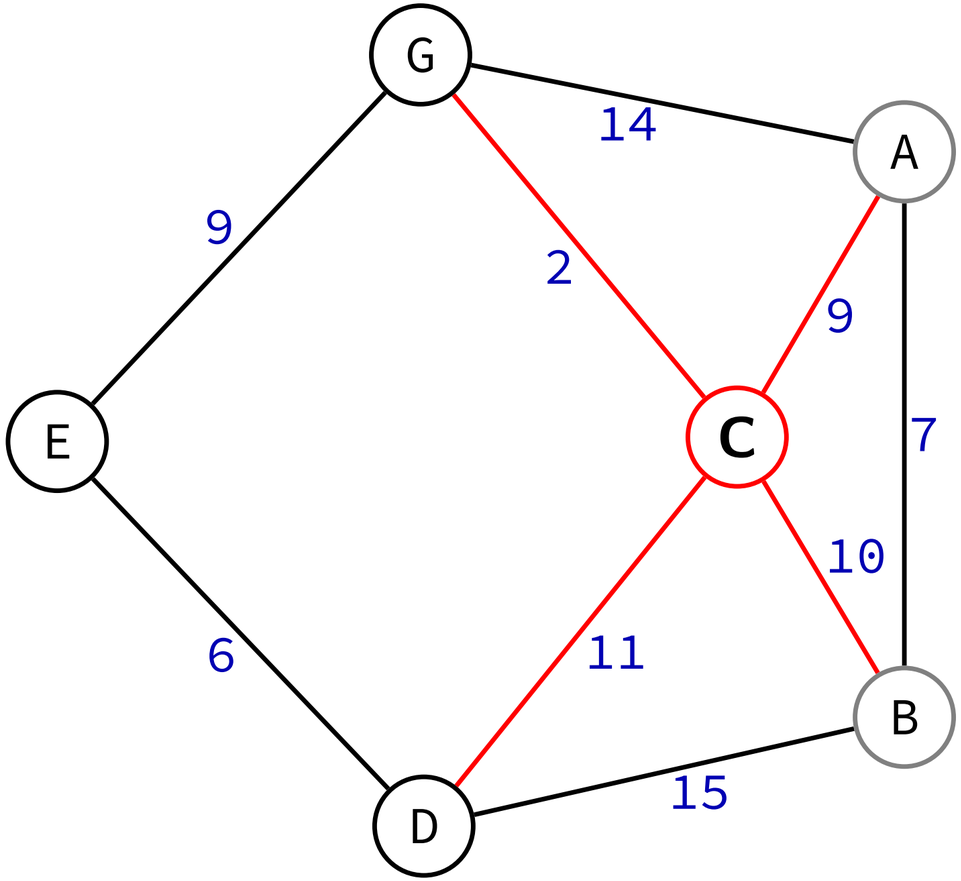

| | dist | prev | path |
|---|---|---|---|
| **A** | 0 | — | A |
| **B** | 7 | A | A→B |
| **C** | 9 | A | A→C |
| D | ∞ | — | — |
| E | ∞ | — | — |
| **G** | 14 | A | A→G |

# Dijkstra's algorithm example 2



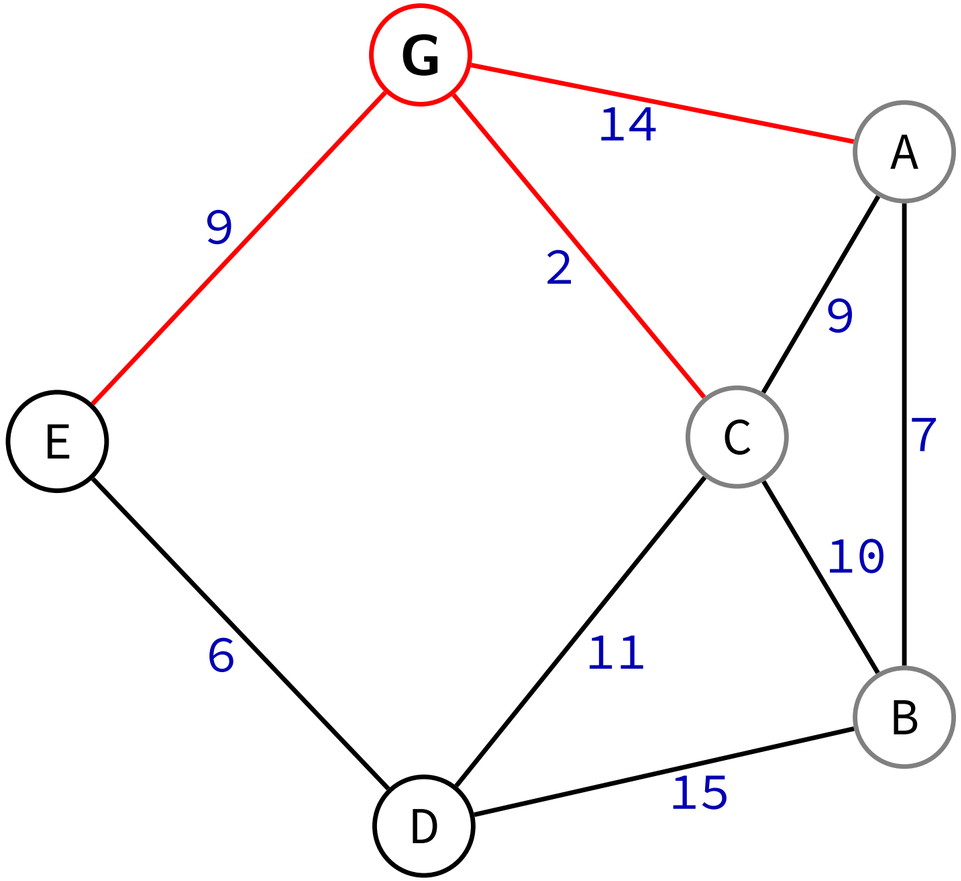

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| **B** | 7 | A | A→B |
| C | 9 | A | A→C |
| **D** | 22 | B | A→B→D |
| E | ∞ | — | — |
| G | 14 | A | A→G |

# Dijkstra's algorithm example 2



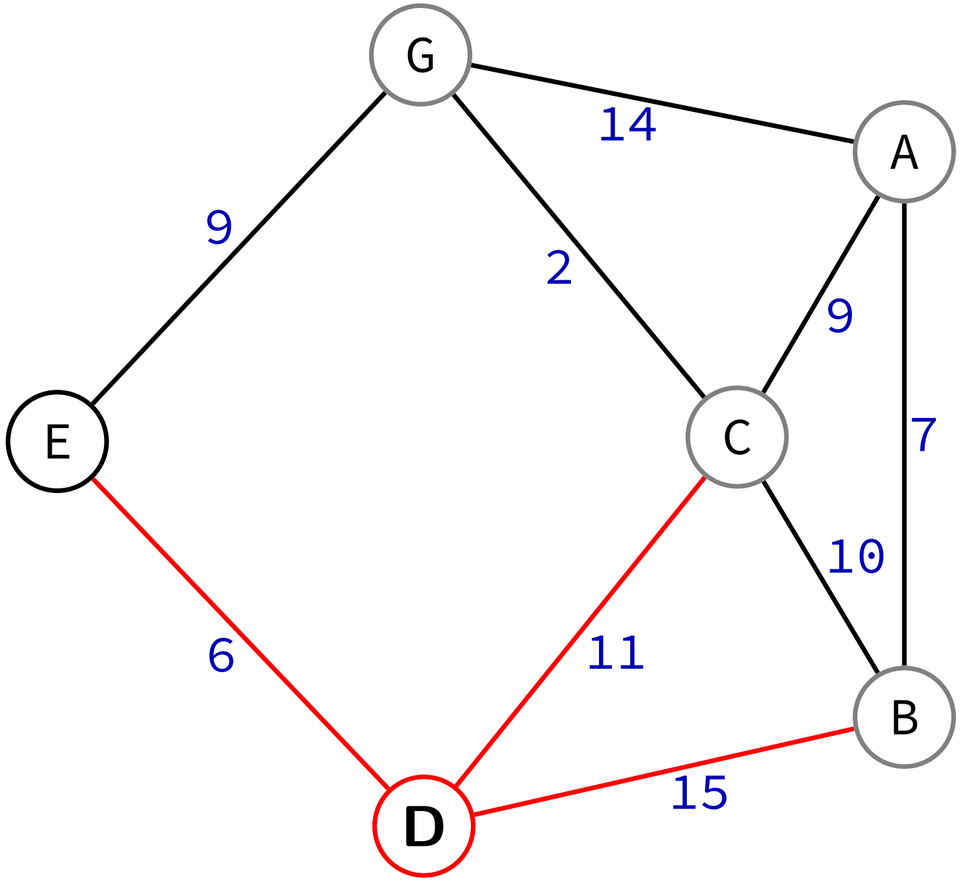

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 7 | A | A→B |
| **C** | 9 | A | A→C |
| **D** | 20 | C | A→C→D |
| E | ∞ | — | — |
| **G** | 11 | C | A→C→G |

# Dijkstra's algorithm example 2



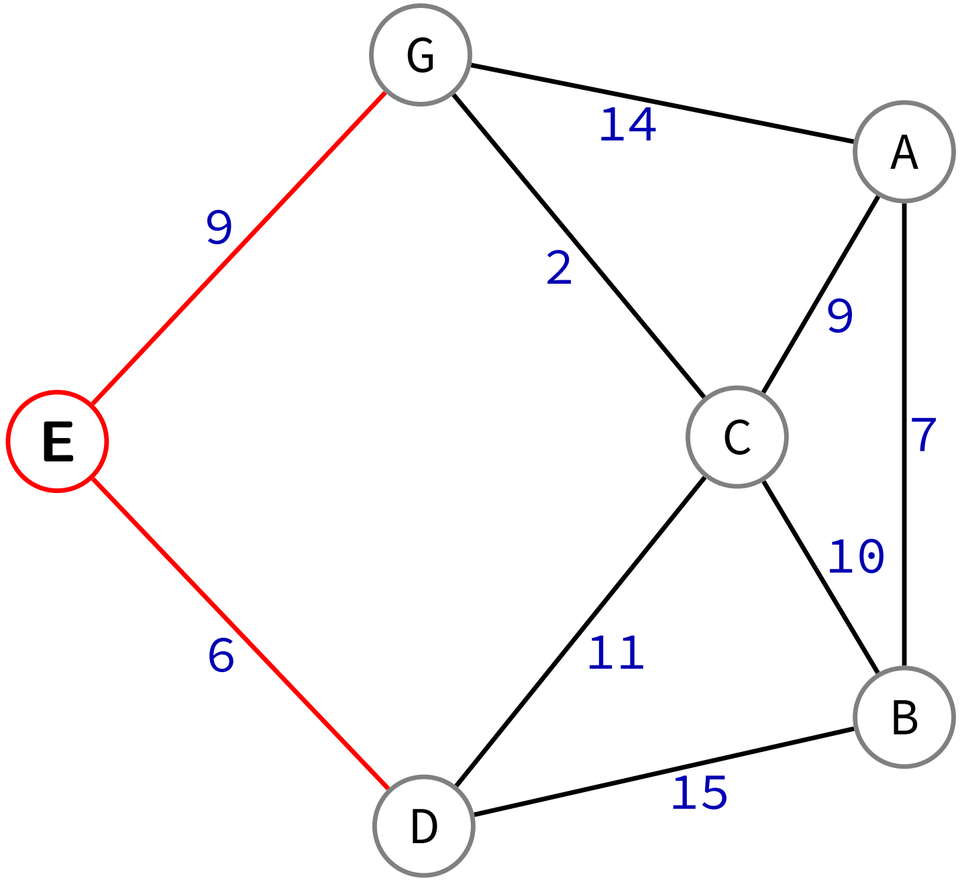

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 7 | A | A→B |
| C | 9 | A | A→C |
| D | 20 | C | A→C→D |
| **E** | 20 | G | A→C→G→E |
| **G** | 11 | C | A→C→G |

# Dijkstra's algorithm example 2



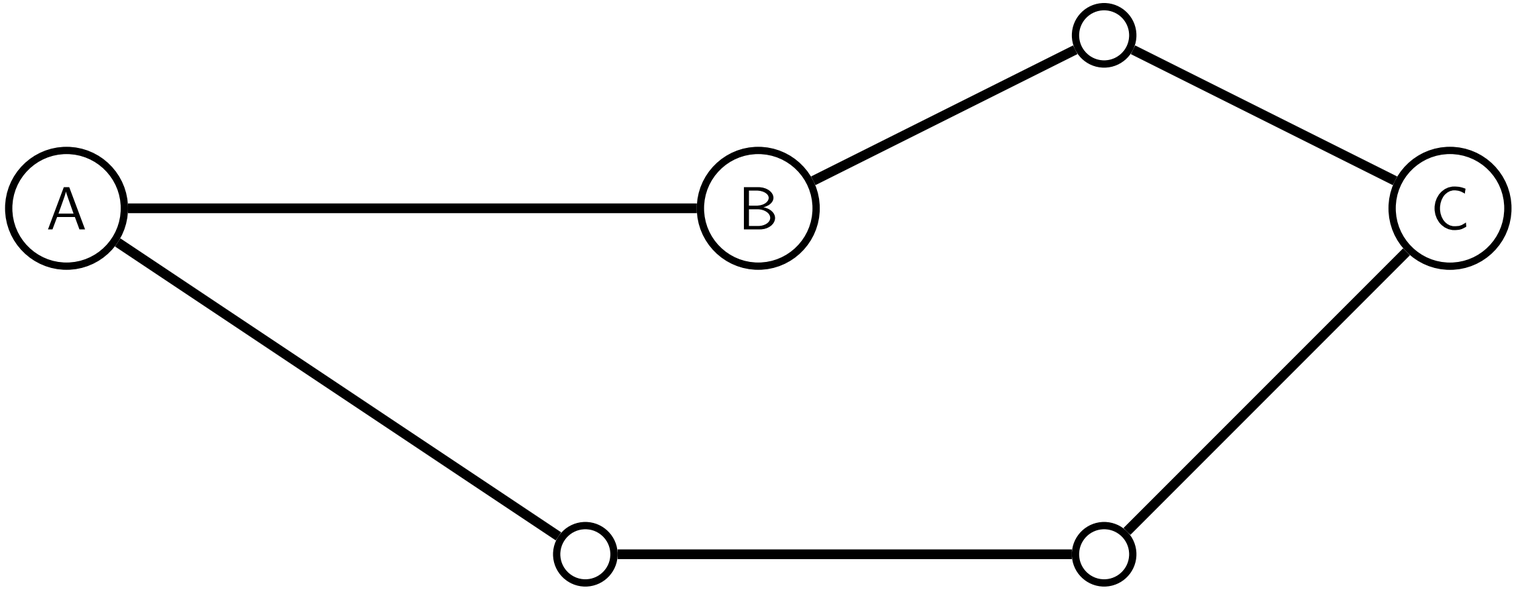

| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 7 | A | A→B |
| C | 9 | A | A→C |
| **D** | 20 | C | A→C→D |
| E | 20 | G | A→C→G→E |
| G | 11 | C | A→C→G |

# Dijkstra's algorithm example 2




| | dist | prev | path |
|---|---|---|---|
| A | 0 | — | A |
| B | 7 | A | A→B |
| C | 9 | A | A→C |
| D | 20 | C | A→C→D |
| **E** | 20 | G | A→C→G→E |
| G | 11 | C | A→C→G |

# consistency

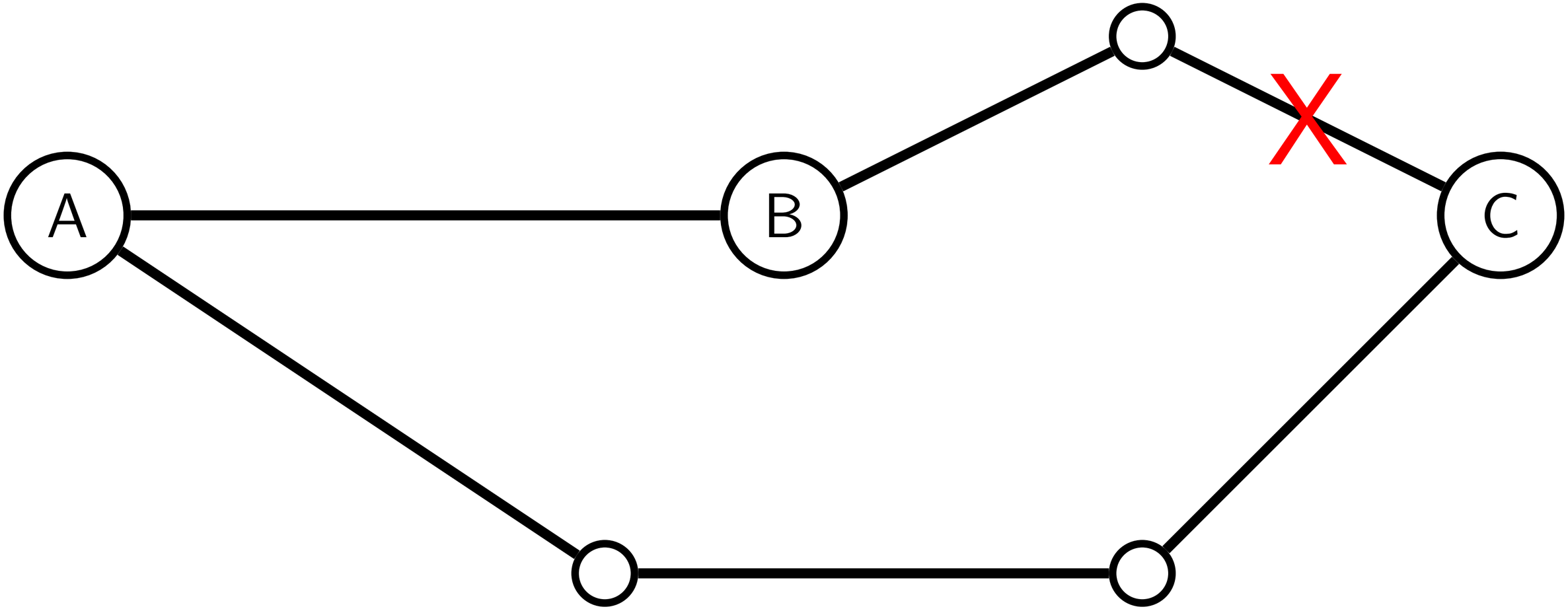



if A sends packet for C to B, how does A know B won't send it back?

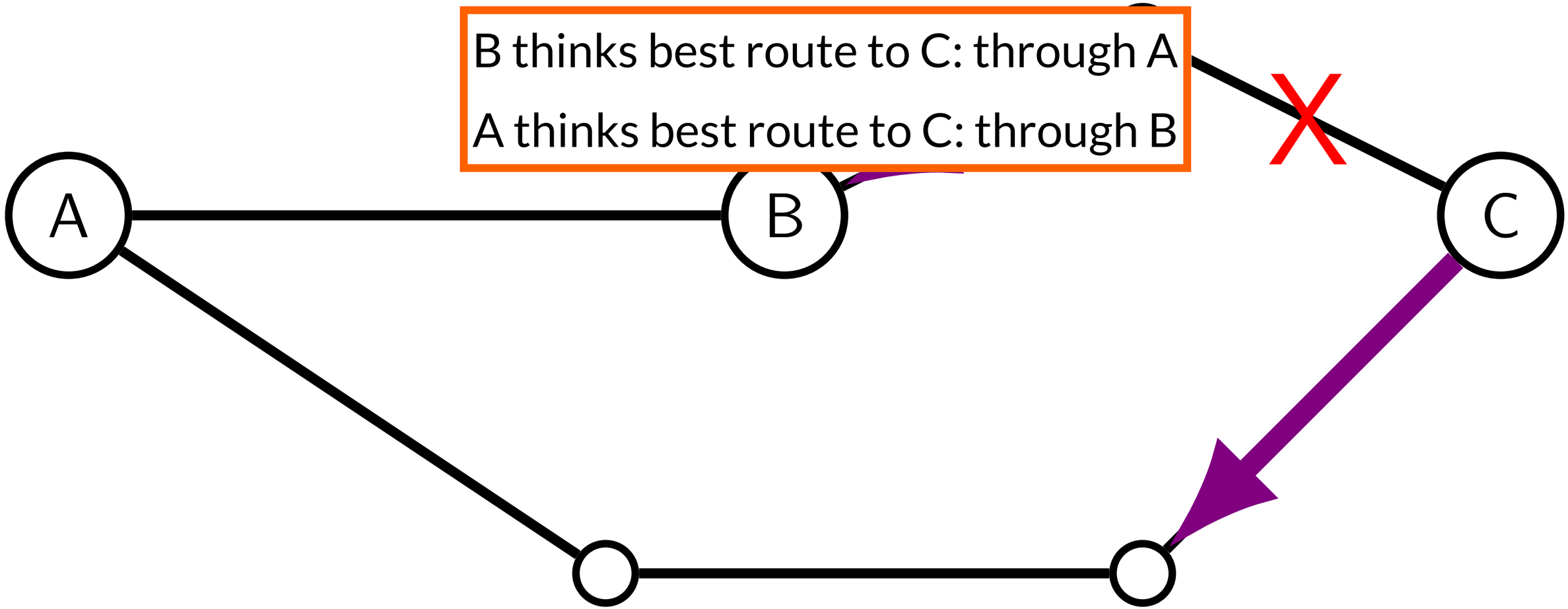
hope: if A thought shortest path to C was through B, then B should agree

problem: not always true

# inconsistency



# inconsistency



# temporary bad routes

while waiting for link state updates to propogate

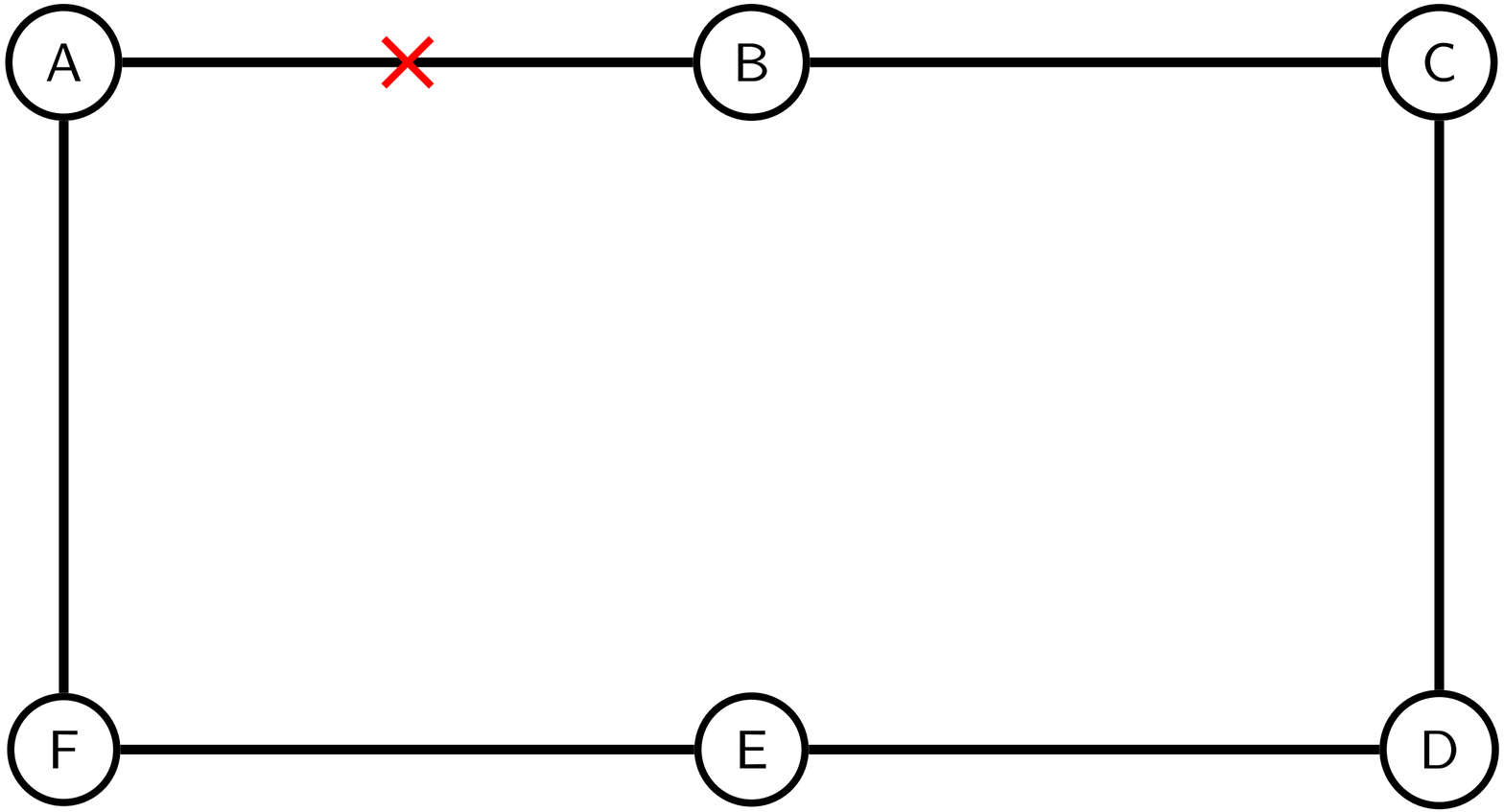
can have too-slow routes

can have routing loops

hope: this is only a few seconds at most

- and routing loop doesn't cause huge explosion of traffic

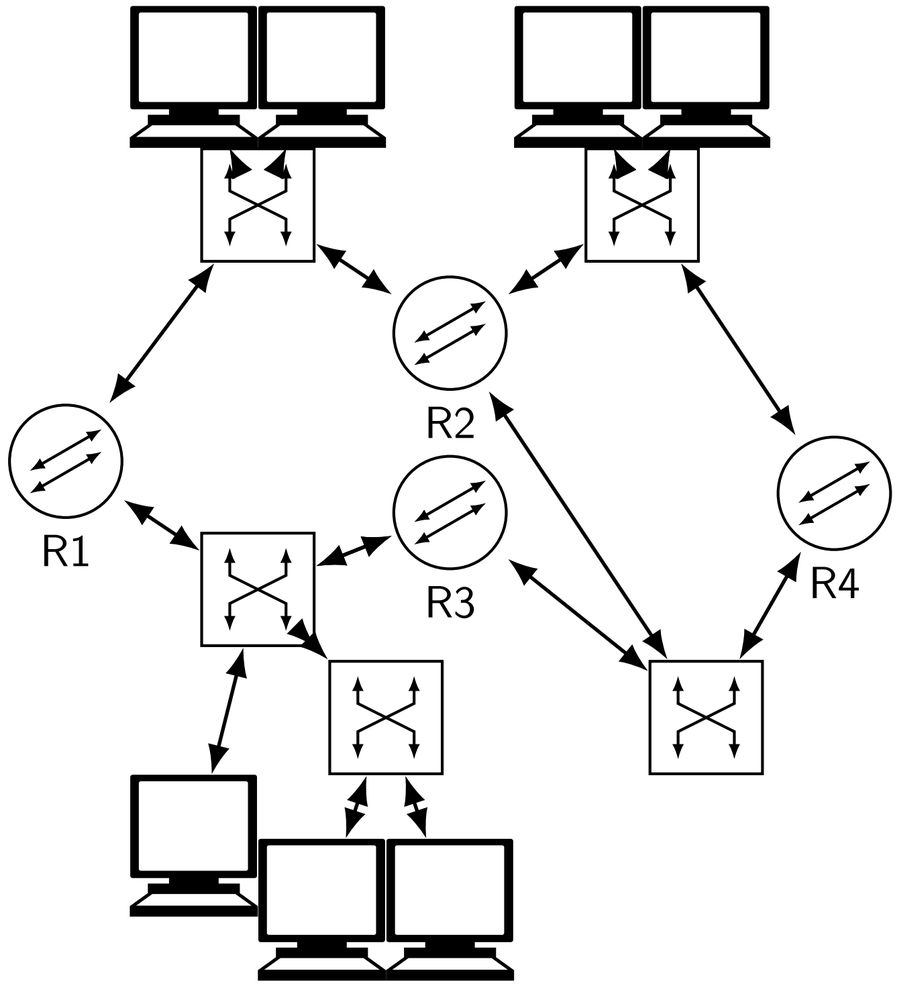
# convergence time



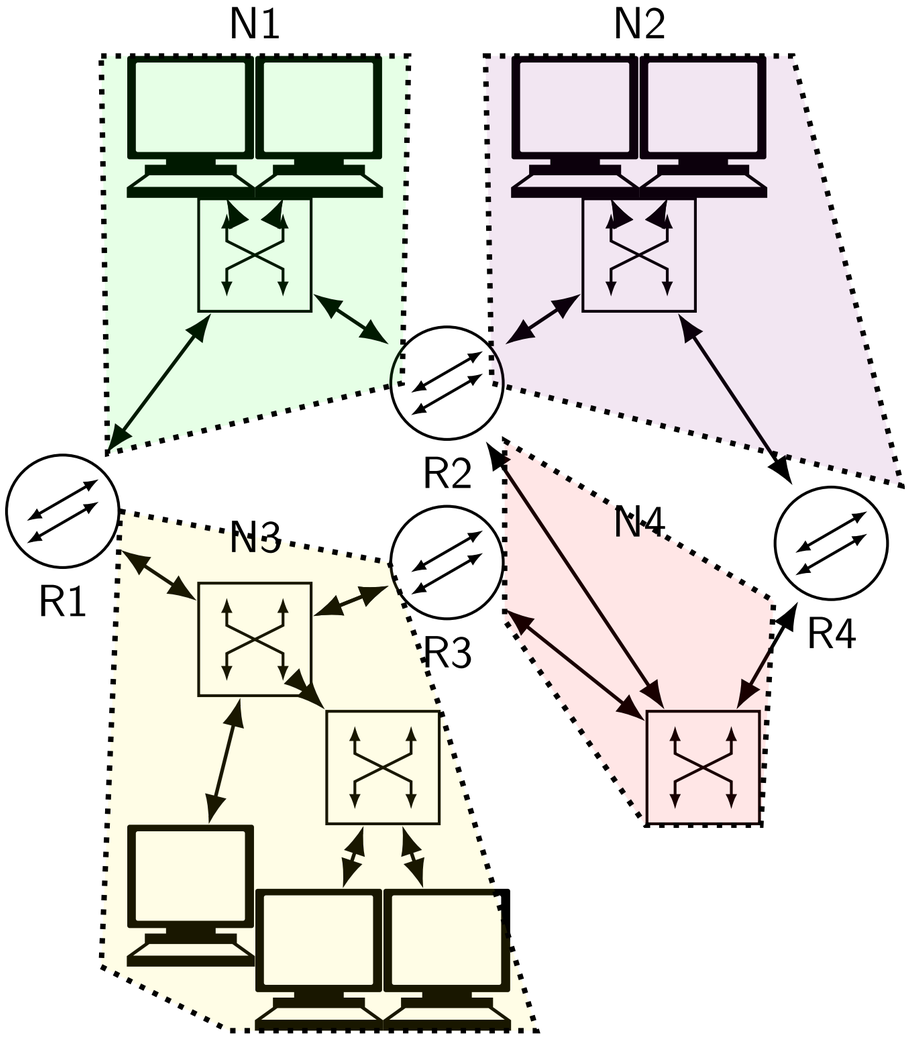

exercise: how many steps to fix A's next hop for B, C, D, etc....

- with distance vector?
- with link state?

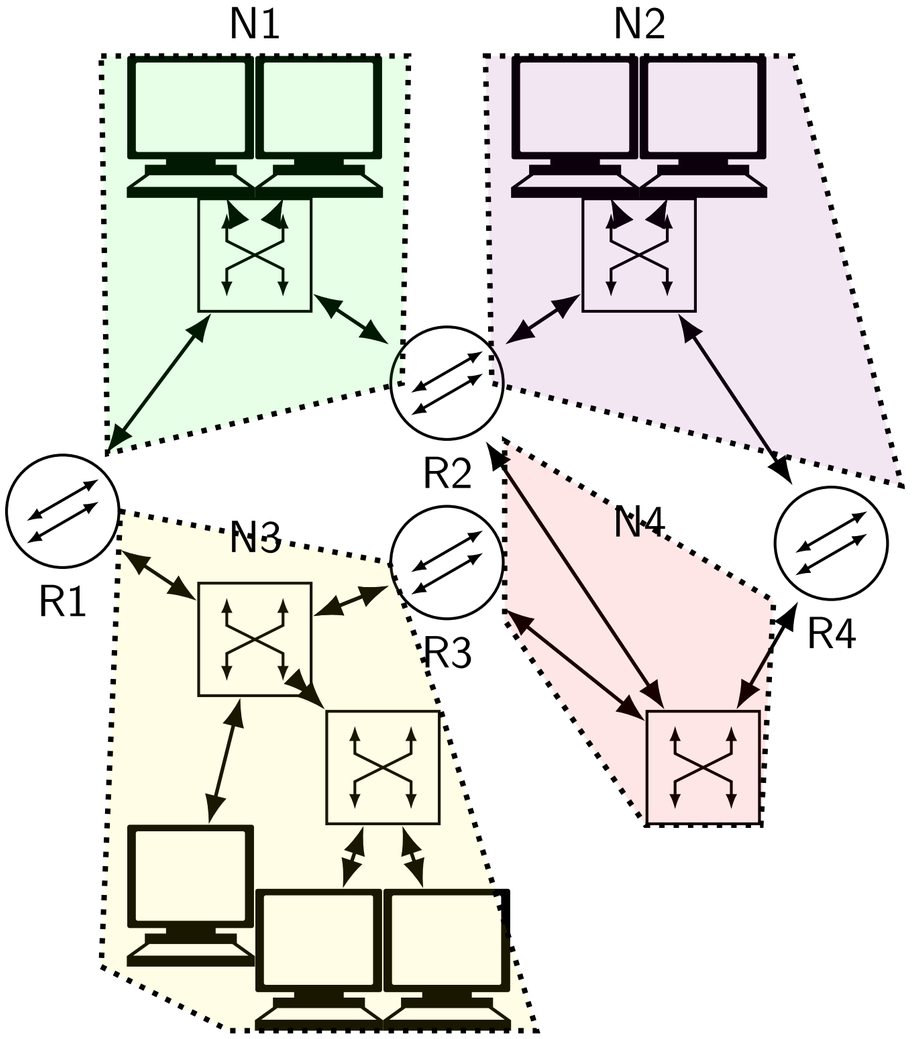
# networks v routers (LS)

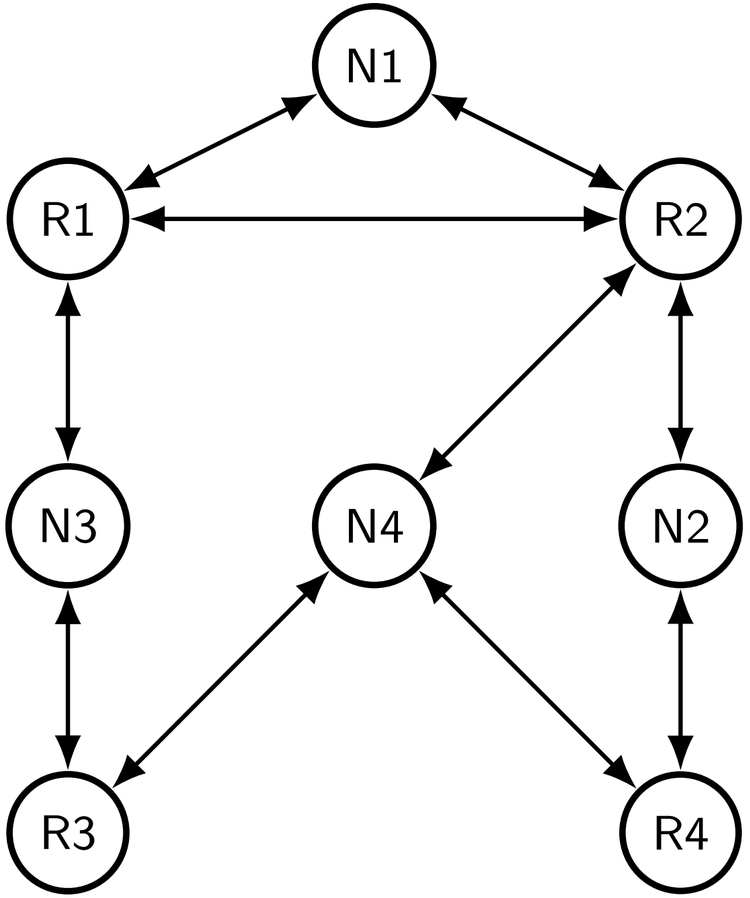


# networks v routers (LS)

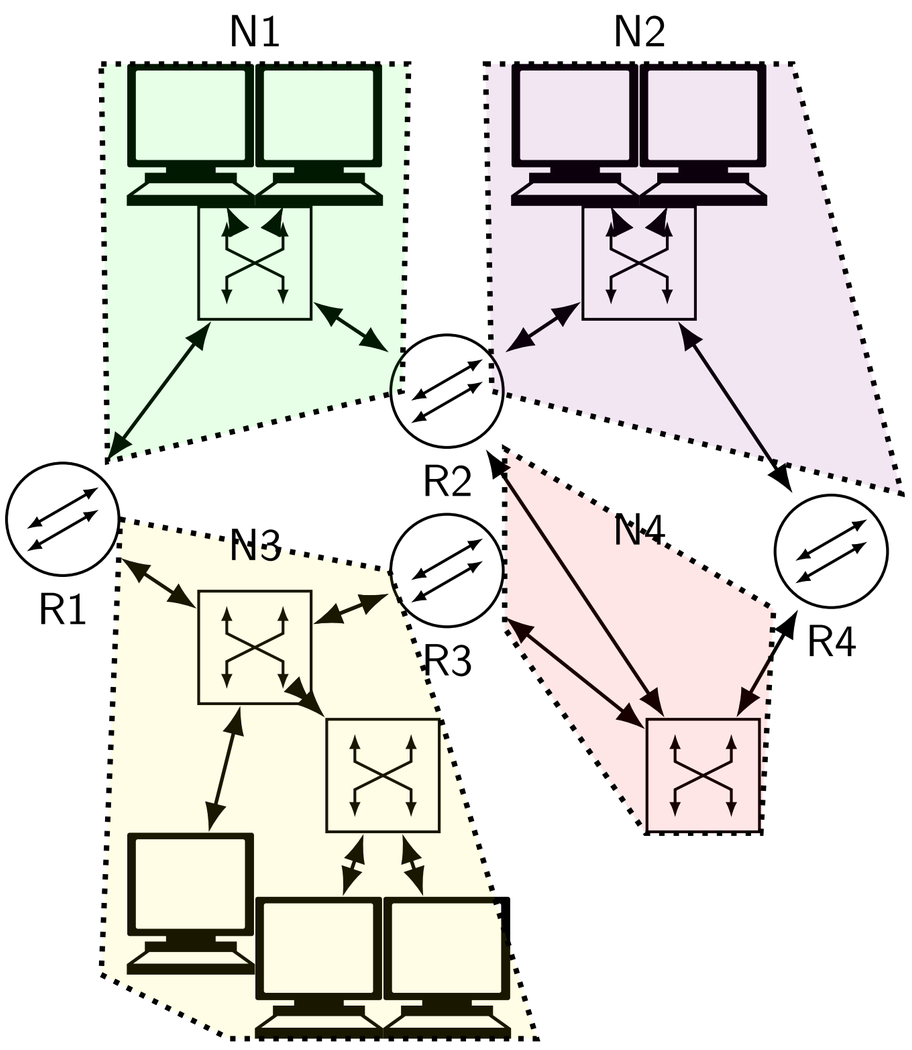


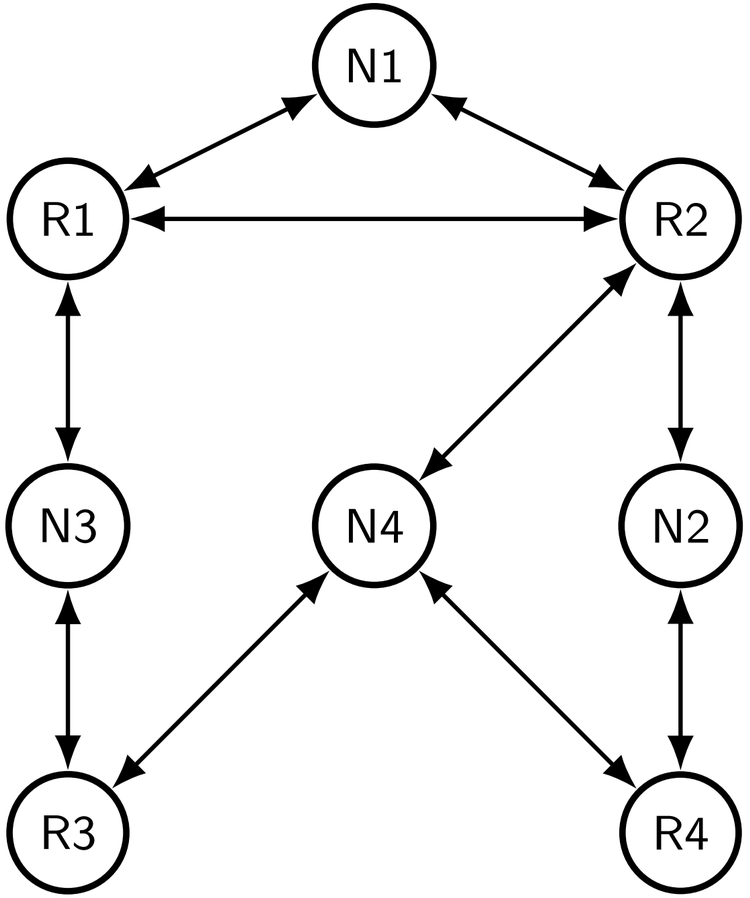
# networks v routers (LS)



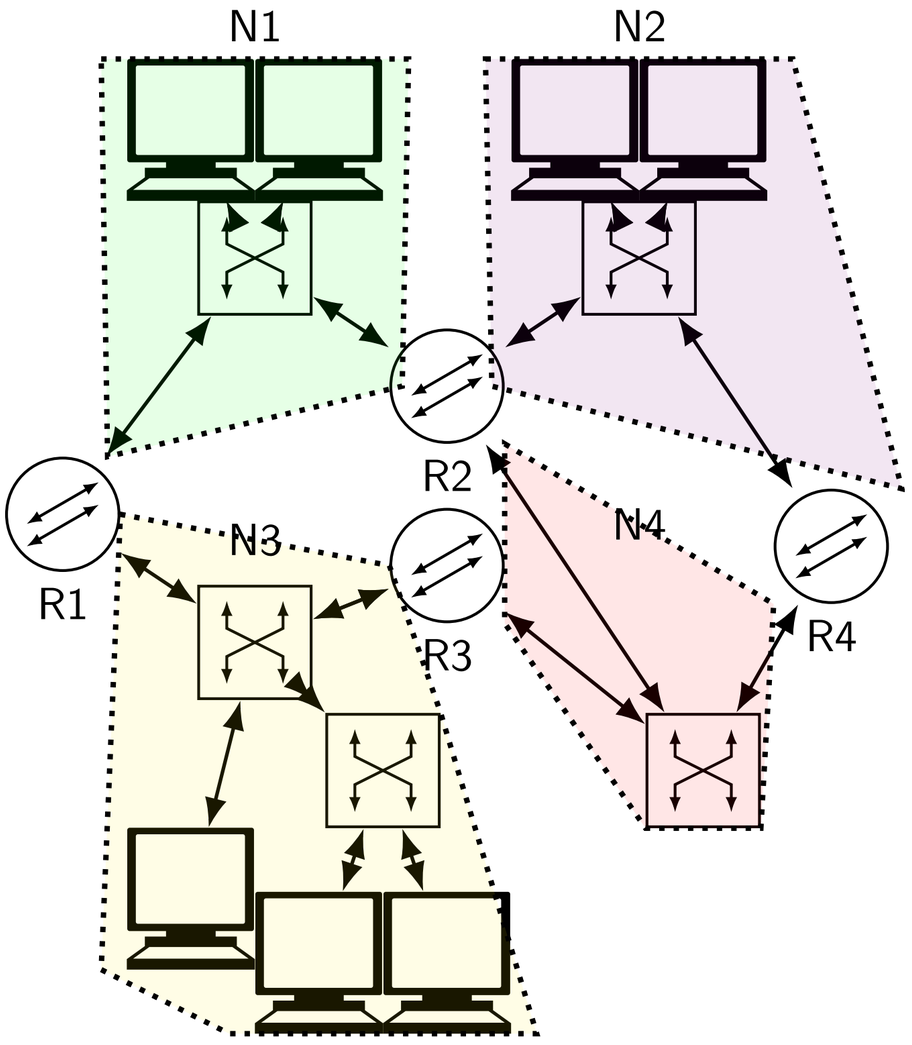
# networks v routers (LS)

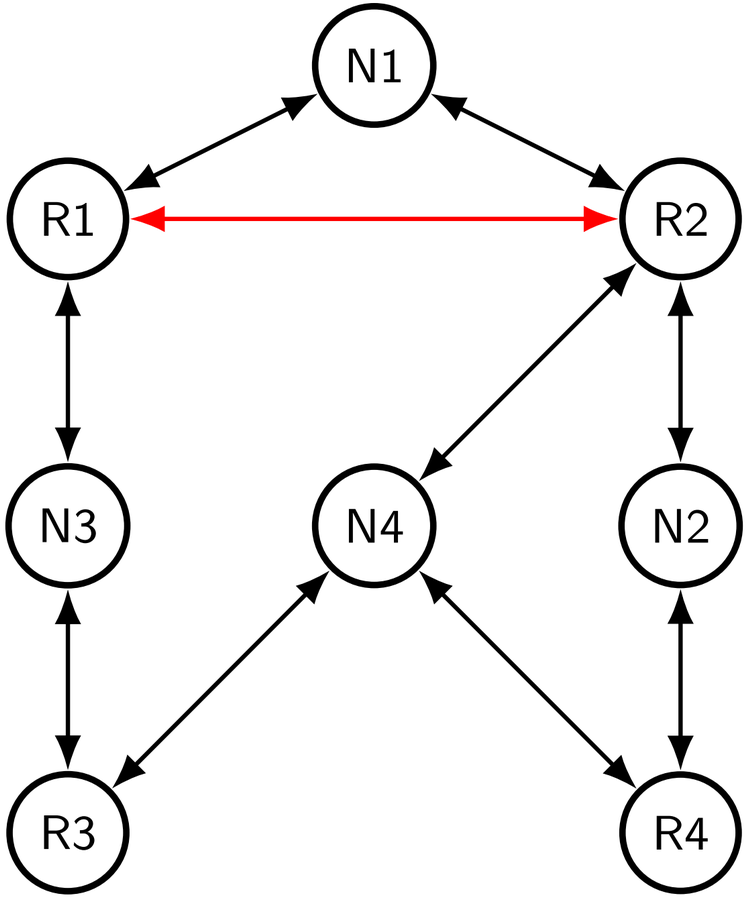



graph has nodes for routers+networks

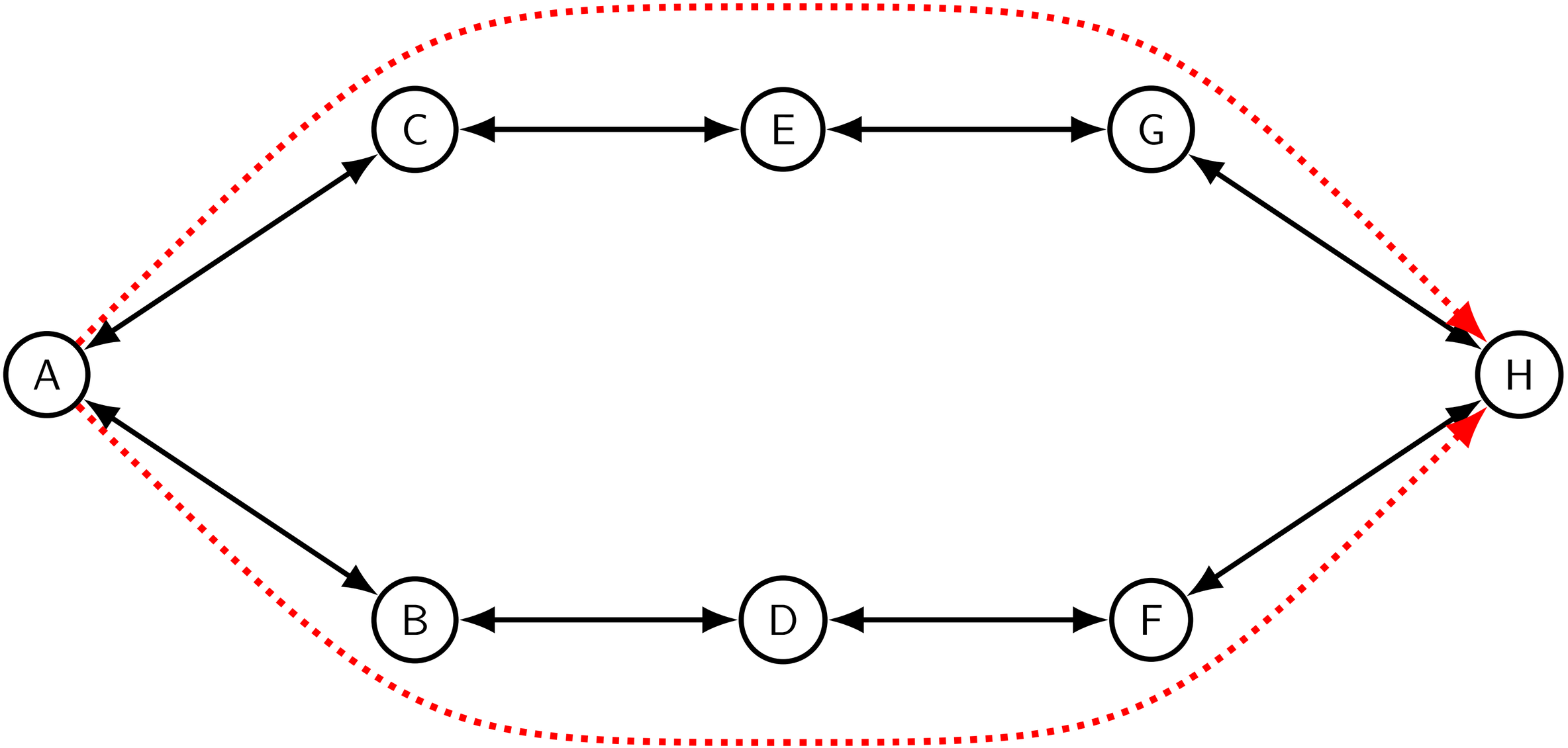
# networks v routers (LS)




can have direct router-router link

# two good choices?



# splitting packets

naïve idea: send every other packet on bottom link

problem: bottom and top link will have different latencies

(even if only temporarily from queuing)

$\Longrightarrow$ packets will be reordered a lot

this is pretty bad for TCP

(and many other things)

# equal cost multipath (ECMP)

split packets by flow

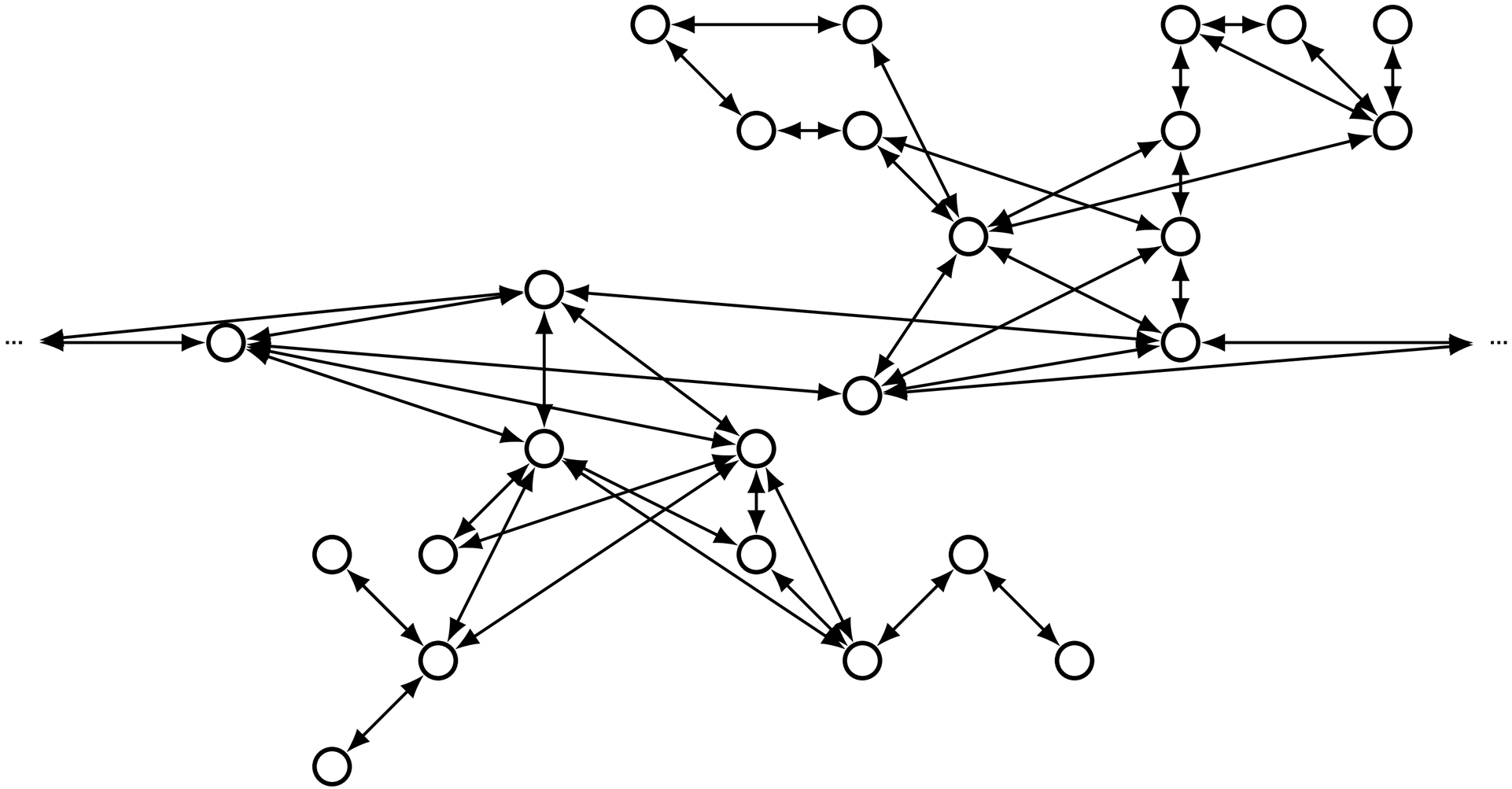
goal:

- each TCP connection chooses one of the $N$ links
- … but don't want to track list of TCP connections

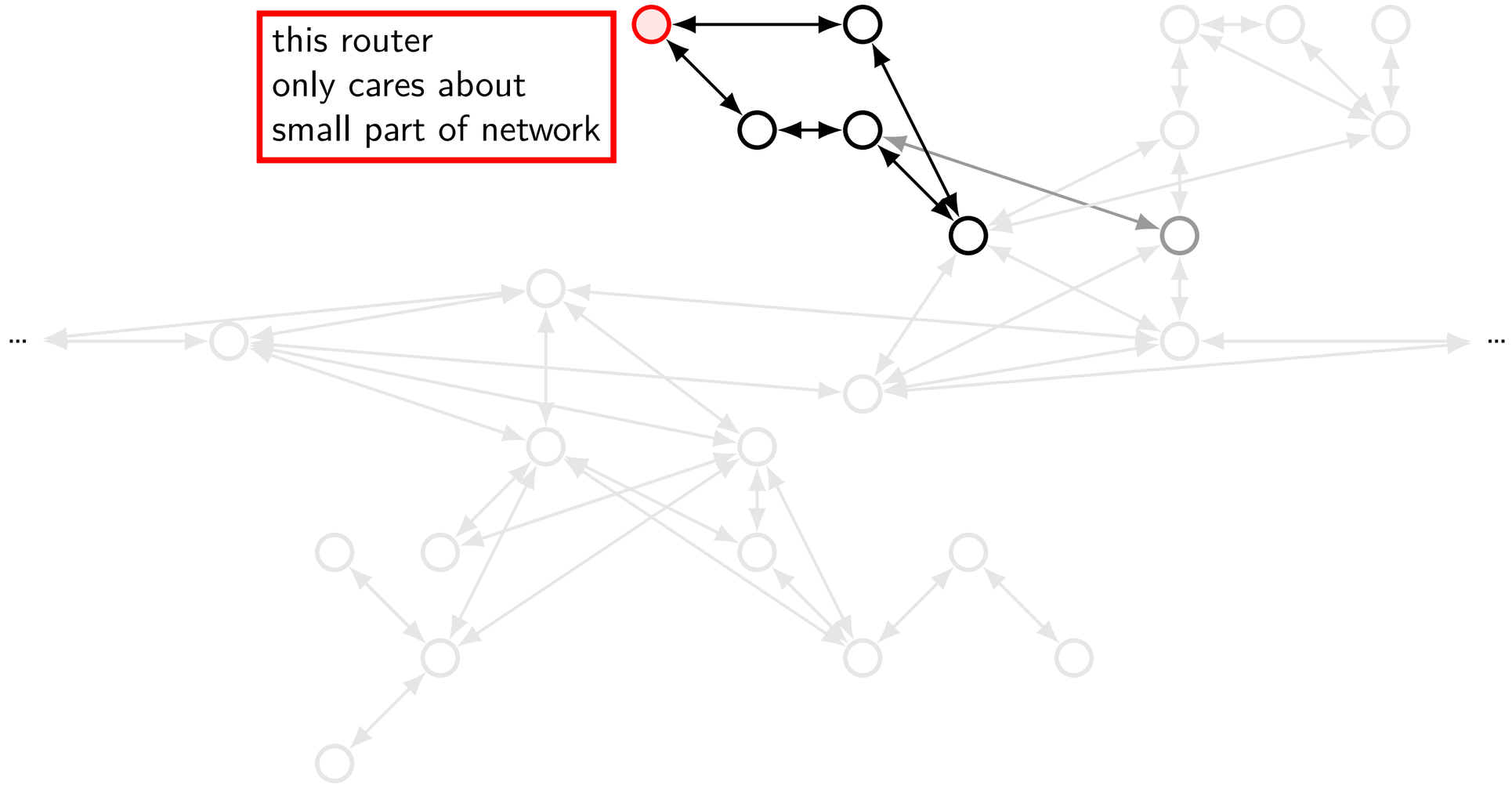
solution:

- take a hash of the connection info in header
- use link index $\left\lfloor \frac{\text{hash value} \times N}{\text{max hash value}} \right\rfloor$

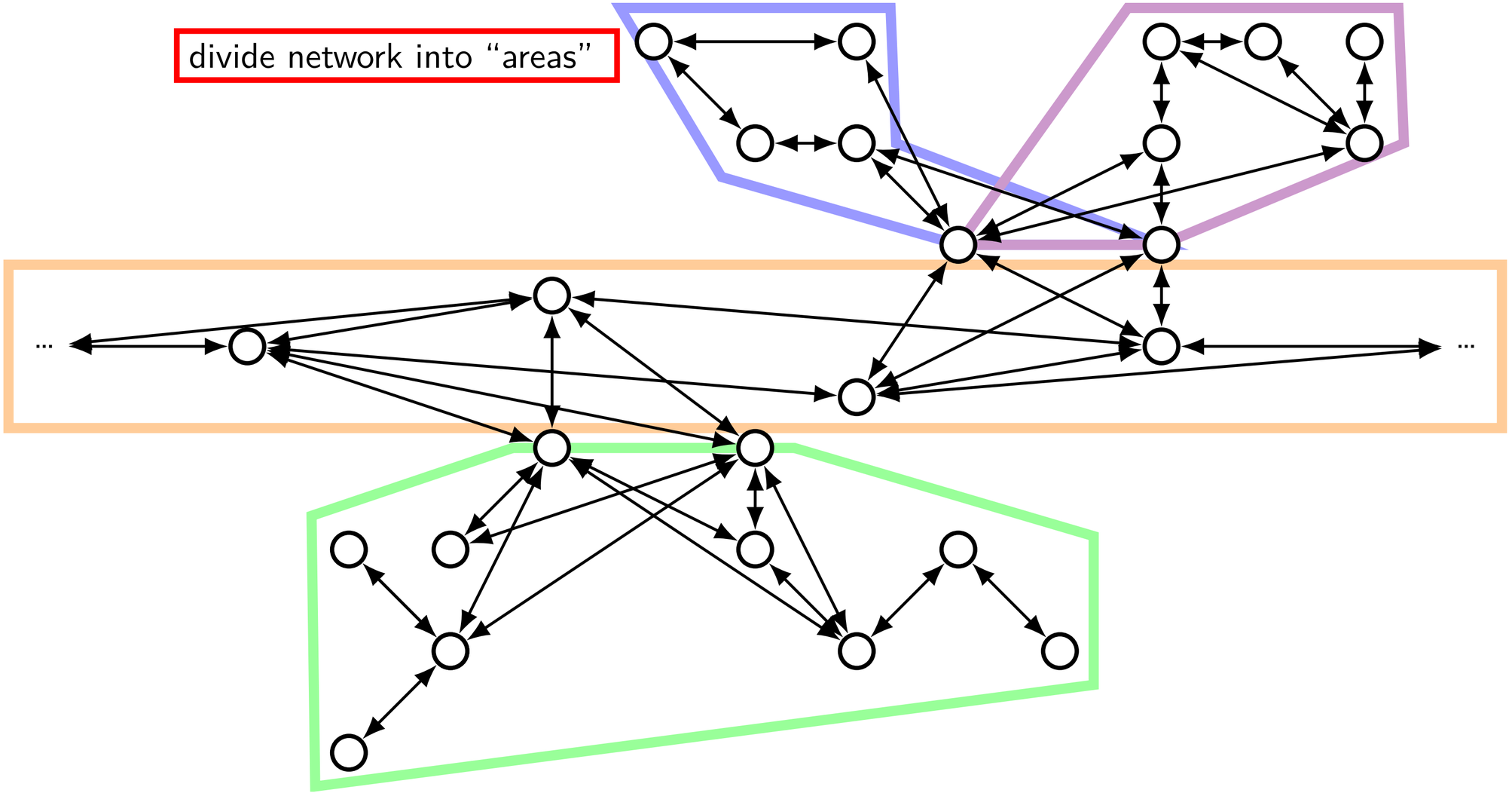
# a big network



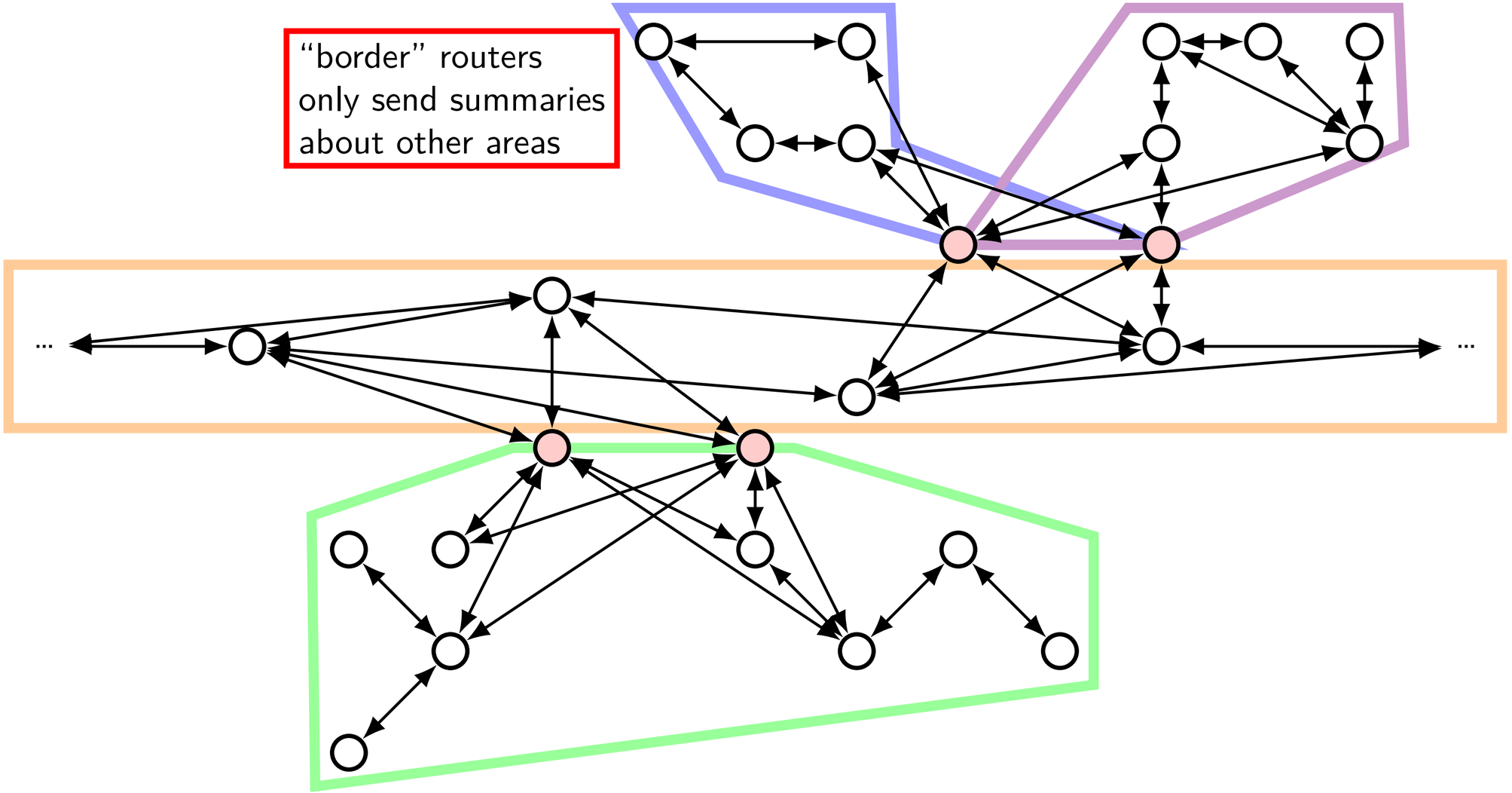
# a big network



# a big network

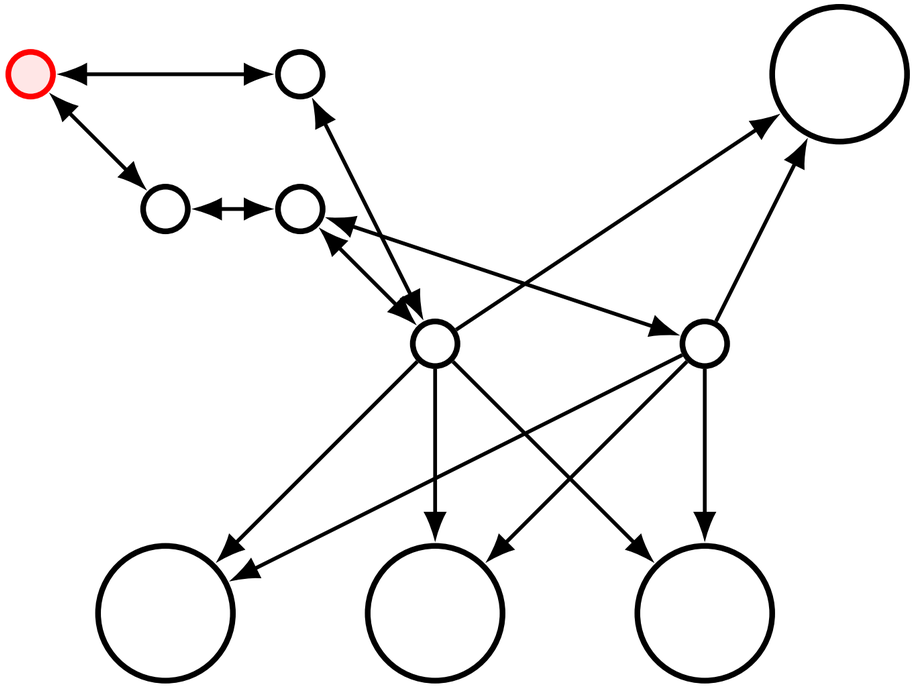


# a big network



# a big network

router sees simpler summary of network
$\rightarrow$ hopefully faster routing



...

...

# distance vector in link state?

summaries are distance vectors!

- area border routers just saying which networks + metric

idea: mix simpler distance vectors with more flexible link-state

# but distance vector problems?

recall: count-to-infinity

let's say areas A, B, C, D all connected to each other…

… and area D goes offline:

could packet for D loop area A to B to C to A to B to C to …

OSPF solution: disallow this network configuration

# backbone

OSPF area 0 is called “backbone”

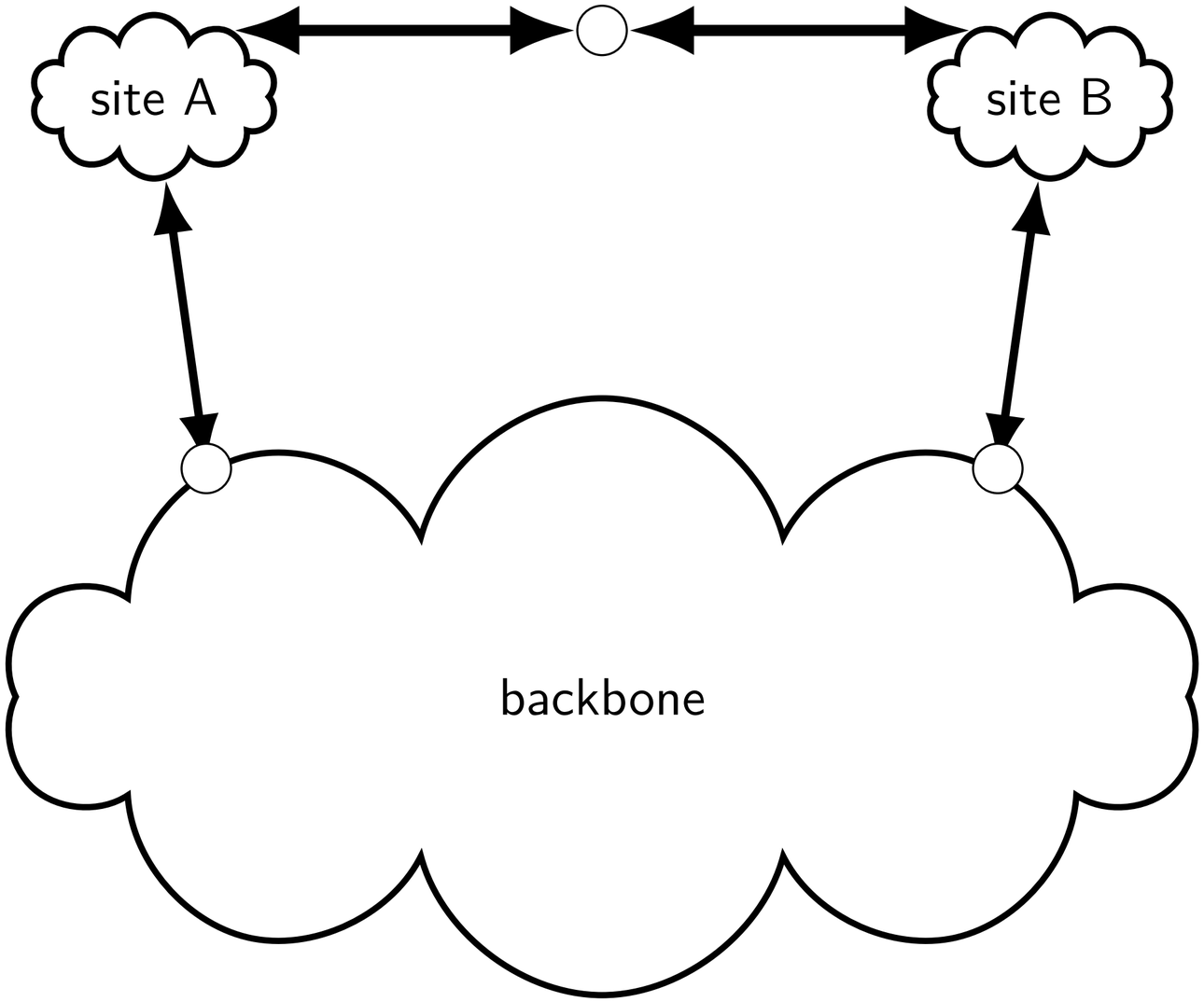
border routers only summarize routes sent to backbone *or* not obtained from other area border routers

means routing between areas must either:

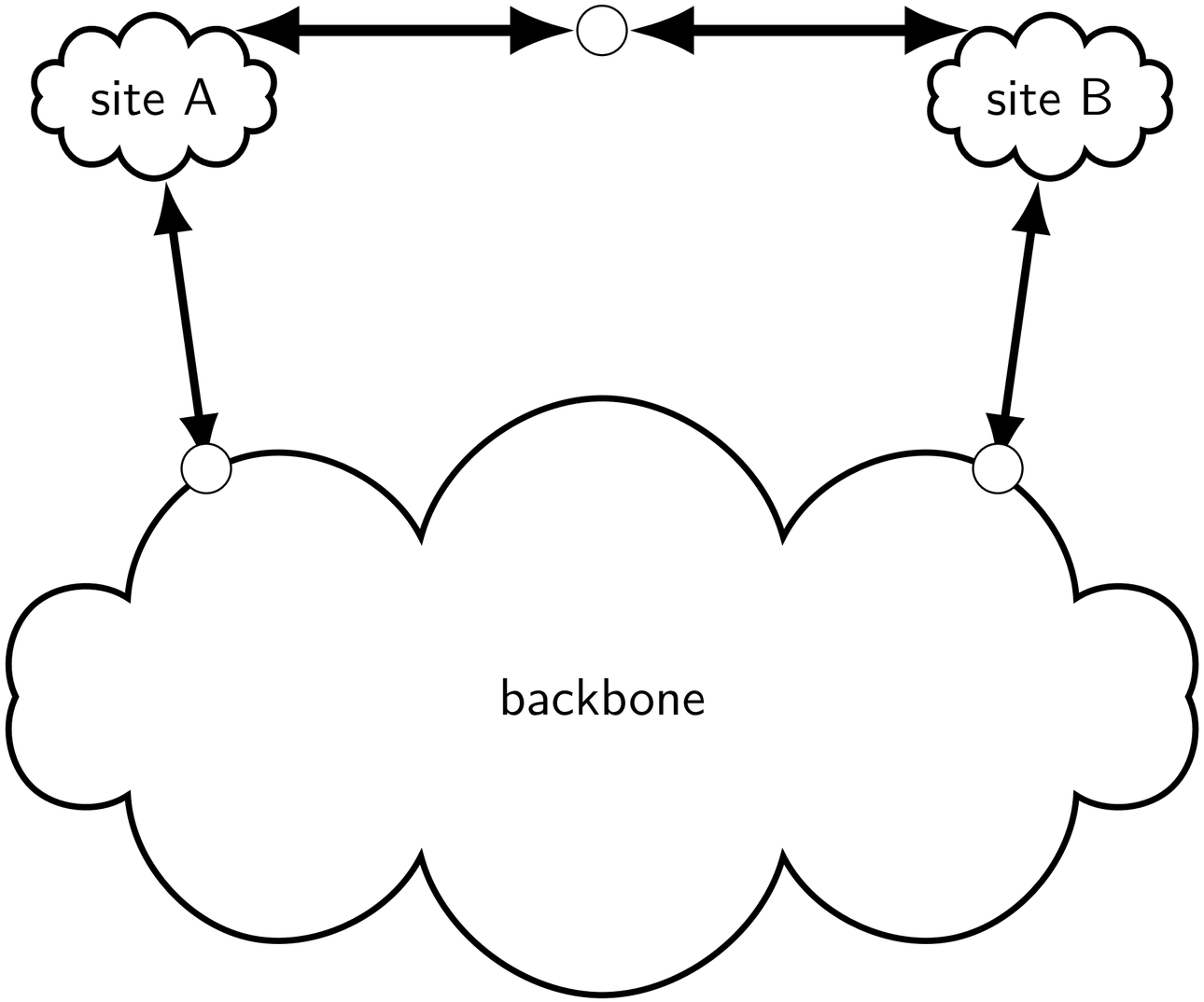
- go through the backbone, or
- only go through one border router

makes loops not possible

# backbone limits

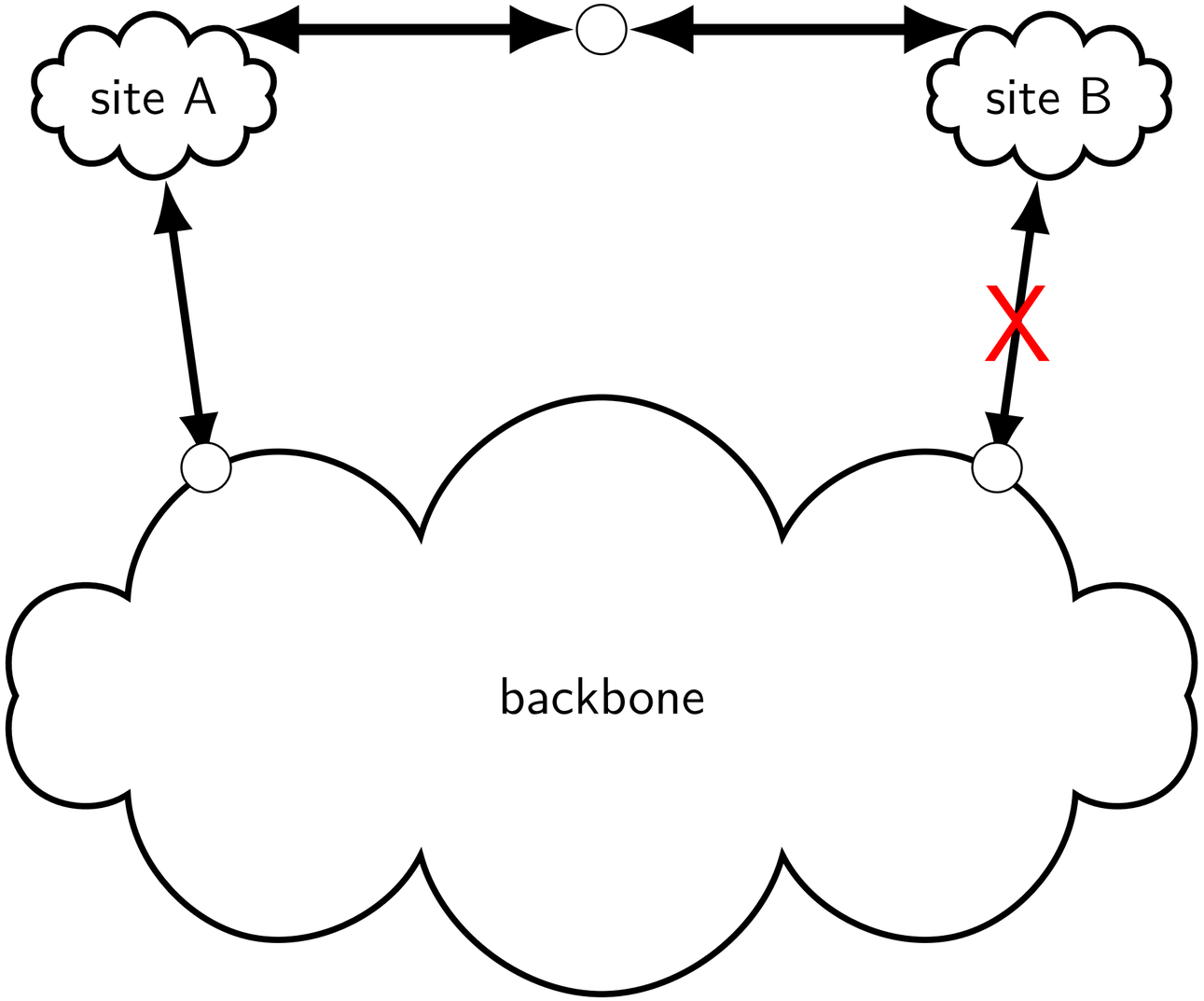


# backbone limits




org with two sites — can configure as areas three border routers

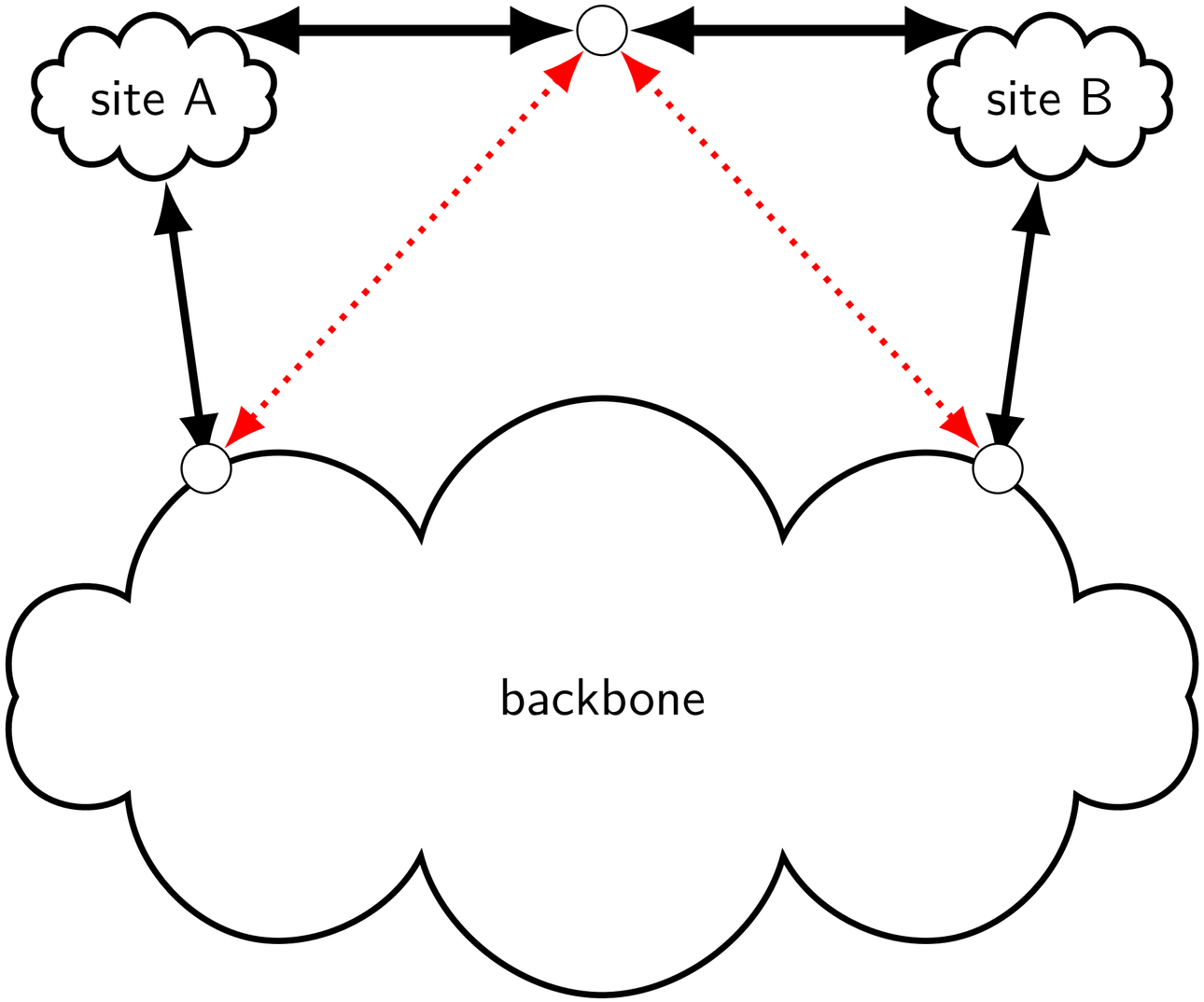
# backbone limits




if B's link to backbone fails, B should use building A's

but disallowed by anti-loop rule

# backbone limits




could fix this
by connecting A/B
border router
to backbone...
solution:
“virtual links”

# OSPF virtual links

“tunnel” backbone through another area

- route as if ‘direct’ connection between two border routers
- but connection implemented by going through area
- both ends considered part of backbone

metric for virtaul link = metric of route through area

configured explicitly by administrator

# backup slides